Centre universitari adscrit a la

TecnoCampus

upf. Universitat Pompeu Fabra Barcelona

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Lab 4

Cryptography & Steganography CTF

Teacher Reference — Do Not Distribute

Do not distribute to students.
This document contains the complete solution including flags and credentials.

Scenario	NovaCorp breach forensic investigation
Servers	ServerA · ServerB · ServerC
Total flags	14 (A1–A5, B1–B5, C1–C4)
Gate	ServerC password = md5(A5 + B5)[:12] = fa681b59855d
Topics	Classical ciphers, AES-ECB, RSA, LSB stego, steghide, ECDSA

Eloi Egea Rada

Supervisor: Pere Vidiella i Catalan

TecnoCampus · GEISI · 2025–2026

Flag Reference (Instructor Summary)

ID	Server	Technique	Flag
A1	ServerA	ROT13	H4U{r0t_c1ph3r_br0k3n}
A2	ServerA	AES-ECB oracle	H4U{4es_3cb_bl0ck_4tt4ck}
A3	ServerA	Vigenere + Kasiski	H4U{v1g3n3r3_k4s1sk1_4tt4ck}
A4	ServerA	XOR repeating key	H4U{x0r_r3p34t1ng_k3y}
A5	ServerA	RSA $e = 3$ cube root	H4U{rsa_sm4ll_3xp0n3nt}
B1	ServerB	EXIF Artist field	H4U{3x1f_m3t4d4t4_h1dd3n}
B2	ServerB	Data after IEND chunk	H4U{d4t4_4ft3r_1end_chunk}
B3	ServerB	LSB red channel	H4U{lsb_st3g4n0gr4phy_r}
B4	ServerB	LSB alpha channel	H4U{4lph4_ch4nn3l_h1dd3n}
B5	ServerB	Steghide JPEG (pass=novacorp)	H4U{st3gh1d3_p4ssw0rd_cr4ck}
C1	ServerC	EXIF hint + steghide + ZIP	H4U{mult1l4y3r_st3g0_4es}
C2	ServerC	Onion: b64(bz2(b64(gz(flag))))	H4U{0n10n_l4y3rs_st3g0}
C3	ServerC	SHA-1 length extension	H4U{l3ngth_3xt3ns10n_sh4}
C4	ServerC	ECDSA nonce reuse secp256k1	H4U{3cc_n0nc3_r3us3_pwn3d}

Table 1: Complete flag reference – instructor use only

Gate Derivation (ServerC Password)

The ServerC password is derived as follows:

```
import hashlib
flag_a5 = "H4U{rsa_sm4ll_3xp0n3nt}"
flag_b5 = "H4U{st3gh1d3_p4ssw0rd_cr4ck}"
combined = flag_a5 + flag_b5
password = hashlib.md5(combined.encode()).hexdigest()[:12]
```

```
# Result: fa681b59855d
```

The devops account password is fa681b59855d. The exercise hint tells students the CTF hint password is D3vOps24! – this is a red herring that hints at the derivation mechanism without giving the answer.

Module A – Cryptography Solutions (ServerA)

Credentials: ssh admin@10.10.1.10 Password: N3xaTech!
Files: /home/admin/mensajes/

A1 – ROT13

Flag: H4U{r0t_c1ph3r_br0k3n}

Solution:

```
import codecs
with open('A1_mensaje_interceptado.txt') as f:
    text = f.read()
decoded = codecs.decode(text, 'rot_13')
print(decoded)  # Flag is embedded in the decoded message
```

ROT13 is a Caesar cipher with a fixed shift of 13. It is self-inverse: applying it twice returns the original text. The flag is embedded in the decoded plaintext.

A2 – AES-ECB Oracle Attack

Flag: H4U{4es_3cb_bl0ck_4tt4ck}

Technique: AES-ECB byte-at-a-time oracle.

AES-ECB encrypts each 16-byte block independently. If the oracle encrypts attacker_input + secret, the attacker can align the unknown bytes one at a time at the end of a block, then brute-force each byte:

```
# Pseudocode for byte-at-a-time ECB attack
recovered = b""
for i in range(len(secret)):
    block_index = i // 16
    padding_len = 15 - (i % 16)
    padding = b"A" * padding_len
    target_block = oracle(padding)[block_index*16:(block_index+1)
       *16]
    for byte in range(256):
        candidate = oracle(padding + recovered + bytes([byte]))
```

```
        if candidate[block_index*16:(block_index+1)*16] ==
           target_block:
            recovered += bytes([byte])
            break
```

Teaching point: ECB mode is deterministic and stateless. Identical plaintext blocks always produce identical ciphertext blocks. This makes it vulnerable to chosen-plaintext attacks in oracle settings. Use CBC or GCM in production.

A3 – Vigenere Cipher + Kasiski Examination

Flag: H4U{v1g3n3r3_k4s1sk1_4tt4ck}
Key: NEXATECH

Solution steps:

1. Find repeated trigrams and their distances. The GCD of distances is 8, indicating a key length of 8.
2. Split the ciphertext into 8 interleaved streams.
3. Apply frequency analysis to each stream as a Caesar cipher.
4. The most frequent letter in each stream corresponds to 'E' in English.

```
from itertools import cycle

def vigenere_decrypt(ciphertext, key):
    result = []
    key_cycle = cycle(key.upper())
    for char in ciphertext:
        if char.isalpha():
            shift = ord(next(key_cycle)) - ord('A')
            base = ord('A') if char.isupper() else ord('a')
            result.append(chr((ord(char) - base - shift) % 26 +
               base))
        else:
            result.append(char)
    return ''.join(result)

plaintext = vigenere_decrypt(open('A3_documento_clasificado.txt').
   read(), 'NEXATECH')
```

A4 – XOR Repeating Key

Flag: H4U{x0r_r3p34t1ng_k3y}

Solution:

```
ciphertext = bytes.fromhex(open('A4_mensaje_xor.hex').read().strip
    ())

# Crib-dragging: try known word 'the' at every position
crib = b'the'
for i in range(len(ciphertext) - len(crib)):
    key_fragment = bytes(c ^ p for c, p in zip(ciphertext[i:], crib
        ))
    # key_fragment[0] corresponds to key[i % key_len]

# Once key length is determined (use index of coincidence), recover
    full key
key_len = 8  # determined by IC analysis
key = bytearray()
for i in range(key_len):
    stream = ciphertext[i::key_len]
    # frequency analysis: most frequent byte XOR ord('e') = key
        byte
    freq = max(range(256), key=lambda k: sum(1 for b in stream if b
        == k))
    key.append(freq ^ ord('e'))

plaintext = bytes(b ^ key[i % len(key)] for i, b in enumerate(
    ciphertext))
```

A5 – RSA Small Exponent ($e = 3$, Cube Root Attack)

Flag: `H4U{rsa_sm4ll_3xp0n3nt}`

Vulnerability: When $e = 3$ and $m^3 < n$, the encryption $c = m^3 \bmod n$ performs no actual modular reduction. Therefore $c = m^3$ as an integer, and $m = \sqrt[3]{c}$ exactly.

```
import gmpy2

# Read from A5_rsa_challenge.txt
n = int(...)   # modulus
e = 3
c = int(...)   # ciphertext

m, exact = gmpy2.iroot(c, e)
assert exact, "Cube␣root␣is␣not␣exact␣--␣modular␣reduction␣occurred
    "

flag = m.to_bytes((m.bit_length() + 7) // 8, 'big').decode()
print(flag)
```

Teaching point: RSA requires a sufficiently large exponent (typically $e = 65537$) or message padding (OAEP) to prevent small-exponent

attacks. Textbook RSA with $e = 3$ and no padding is broken when the plaintext is small relative to the modulus.

Module B – Steganography Solutions (ServerB)

Credentials: `ssh sysop@10.10.2.10` Password: `S3cure24!`
Files: `/srv/public/uploads/`

B1 – EXIF Metadata

Flag: `H4U{3x1f_m3t4d4t4_h1dd3n}`

```
exiftool B1_oficina_novacorp.png
# Look for: Artist: H4U{3x1f_m3t4d4t4_h1dd3n}
```

The flag is stored in the EXIF `Artist` field. EXIF metadata is invisible to ordinary image viewers but trivially readable with `exiftool`.

B2 – Data after PNG IEND Chunk

Flag: `H4U{d4t4_4ft3r_1end_chunk}`

```
with open('B2_network_diagram.png', 'rb') as f:
    data = f.read()

# PNG IEND chunk signature + CRC (8 bytes total)
iend_marker = b'\x49\x45\x4e\x44\xae\x42\x60\x82'
pos = data.find(iend_marker)
hidden = data[pos + len(iend_marker):]
print(hidden.decode())  # H4U{d4t4_4ft3r_1end_chunk}
```

PNG parsers stop reading at IEND. Any appended bytes are silently ignored by image viewers, making this a trivial but effective hiding technique.

B3 – LSB Steganography in Red Channel

Flag: `H4U{lsb_st3g4n0gr4phy_r}`

```
from PIL import Image

img = Image.open('B3_documento_escaneado.png').convert('RGB')
bits = []
for y in range(img.height):
    for x in range(img.width):
        r, g, b = img.getpixel((x, y))
        bits.append(r & 1)
```

```
chars = []
for i in range(0, len(bits), 8):
    byte = int(''.join(str(b) for b in bits[i:i+8]), 2)
    if byte == 0:
        break
    chars.append(chr(byte))

print(''.join(chars))  # H4U{lsb_st3g4n0gr4phy_r}
```

B4 – LSB Steganography in Alpha Channel

Flag: H4U{4lph4_ch4nn3l_h1dd3n}

```
from PIL import Image

img = Image.open('B4_novacorp_logo.png').convert('RGBA')
alpha = img.split()[3]   # Channel index 3 = Alpha
bits = []
for y in range(img.height):
    for x in range(img.width):
        bits.append(alpha.getpixel((x, y)) & 1)

chars = []
for i in range(0, len(bits), 8):
    byte = int(''.join(str(b) for b in bits[i:i+8]), 2)
    if byte == 0:
        break
    chars.append(chr(byte))

print(''.join(chars))  # H4U{4lph4_ch4nn3l_h1dd3n}
```

B5 – Steghide JPEG (Passphrase: novacorp)

Flag: H4U{st3gh1d3_p4ssw0rd_cr4ck}
Passphrase: novacorp

```
steghide extract -sf B5_camara_seguridad.jpg -p novacorp
# Outputs: flag.txt containing H4U{st3gh1d3_p4ssw0rd_cr4ck}
```

The passphrase is the company name in lowercase. In a real assessment, this would be found via wordlist attack using `stegseek` or `steghide` with `rockyou.txt`.

Module C – Advanced Mix Solutions (ServerC)

Credentials: `ssh devops@10.10.3.10` Password: `fa681b59855d`
Files: `/home/devops/retos/`

C1 – Multilayer: EXIF + Steghide + ZIP

Flag: H4U{mult1l4y3r_st3g0_4es}

Step 1: Extract EXIF comment for steghide passphrase.

```
exiftool C1_backup_tapes.jpg
# Comment: passphrase=cipherstrike2026
```

Step 2: Extract steghide payload.

```
steghide extract -sf C1_backup_tapes.jpg -p cipherstrike2026
# Extracts: payload.zip
```

Step 3: Open ZIP archive.

```
unzip payload.zip
cat flag.txt   # H4U{mult1l4y3r_st3g0_4es}
```

Implementation note: The `piexif` library must re-insert the EXIF comment *after* steghide embedding, as steghide strips EXIF on embed. The generator script uses `piexif.insert(exif_bytes, filename)` after `steghide embed` to restore the hint metadata.

C2 – Onion Encoding: b64(bz2(b64(gz(flag))))

Flag: H4U{0n10n_l4y3rs_st3g0}

```
import base64, gzip, bz2

with open('C2_backup_log.b64') as f:
    data = f.read().strip()

# Layer 1: base64 decode
data = base64.b64decode(data)
# Layer 2: bzip2 decompress
data = bz2.decompress(data)
# Layer 3: base64 decode
data = base64.b64decode(data)
# Layer 4: gzip decompress
data = gzip.decompress(data)

print(data.decode())  # H4U{0n10n_l4y3rs_st3g0}
```

Note: the encoding order from inside out is gz then b64 then bz2 then b64, so the decoding order is b64 then bz2 then b64 then gz.

C3 – SHA-1 Hash Length Extension Attack

Flag: H4U{l3ngth_3xt3ns10n_sh4}

Vulnerability: The server computes `MAC = SHA1(secret + message)`. SHA-1 (like MD5) uses Merkle-Damgard construction, which allows an attacker who knows the MAC to append data and compute a valid MAC for the extended message without knowing the secret.

```
import hlextend

# From C3_api_token.txt:
original_msg = b"user=analyst&role=viewer"
original_mac = "a3f......"  # 40-hex SHA1 from the file
secret_len = 16             # hinted in the challenge

sha = hlextend.new('sha1')
# Forge: extend with &access=admin
forged_msg = sha.extend(b'&access=admin', original_msg, secret_len,
    original_mac)
forged_mac = sha.hexdigest()

# Submit to verificator
# python3 C3_verificador.py <forged_msg_hex> <forged_mac>
import subprocess
result = subprocess.run(
    ['python3', 'C3_verificador.py', forged_msg.hex(), forged_mac],
    capture_output=True, text=True
)
print(result.stdout)  # H4U{l3ngth_3xt3ns10n_sh4}
```

> **Teaching point:** `SHA1(secret + message)` is not a secure MAC. Use HMAC-SHA256 instead. HMAC wraps the hash with two nested applications that prevent length extension by design.

C4 – ECDSA Nonce Reuse (secp256k1)

Flag: `H4U{3cc_n0nc3_r3us3_pwn3d}`

Vulnerability: ECDSA security requires a unique random nonce k for every signature. If the same k is reused across two signatures (r, s_1, h_1) and (r, s_2, h_2) (note: same r implies same k), the private key is algebraically recoverable.

Recovery equations:

$$k = \frac{h_1 - h_2}{s_1 - s_2} \pmod{n} \qquad d = \frac{s_1 \cdot k - h_1}{r} \pmod{n}$$

```
from sympy import mod_inverse

# secp256k1 curve order
n = 0
   xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
```

```
# Read from C4_ecdsa_signatures.txt -- find two entries with same r
# (r, s1, h1) and (r, s2, h2) share the same k

def modinv(a, m):
    return pow(a, -1, m)

k = (h1 - h2) * modinv(s1 - s2, n) % n
d = (s1 * k - h1) * modinv(r, n) % n

print(f"Recovered private key: {hex(d)}")

# Sign the challenge message with d
# Submit r_hex s_hex to C4_verificador.py
```

Real-world precedent: The Sony PlayStation 3 root key was recovered in 2010 using exactly this attack. Sony's ECDSA implementation used a constant k for all firmware signatures.

Grading Guide

Item	Criteria	Points
Flags A1–A4	Correct flag strings (1 pt each)	4
Flag A5	Correct flag + gate derivation shown	3
Flags B1–B4	Correct flag strings (1 pt each)	4
Flag B5	Correct flag + gate derivation shown	3
Flags C1–C4	Correct flag strings (2 pts each)	8
Methods	Clear explanation of technique per flag	6
Code quality	Working scripts submitted, readable, commented	4
Reflection	Substantive paragraph on difficulty/learning	2
Bonus	C4 with detailed algebraic derivation	+2
Total		**34 (+2)**

Table 2: Grading rubric